

Algorithmen und Datenstrukturen

Übung 1 & 2: Grundlagen

Robert

2026-05-08

Wiederholung Algorithmen

Anforderungen an Algorithmen

1. **Allgemeinheit:**

- ▶ Kann der Algorithmus auf eine Gruppe von Problemen angewendet werden und nicht nur auf ein spezifisches Problem?

2. **Determiniertheit:**

- ▶ Kommt bei gleicher Eingabe immer die gleiche Ausgabe heraus?

3. **Determinismus:**

- ▶ Führt der Algorithmus bei der gleichen Eingabe auch immer die gleichen Schritte aus?

4. **(partielle) Korrektheit:**

- ▶ Liefert der Algorithmus das richtige Ergebnis (und terminiert er)?

5. **Effizienz:**

- ▶ Nutzt der Algorithmus keine unnötigen Ressourcen (Zeit, Speicher)?

Aufgabe 1

1.1.a

```
public static boolean f0(int a) {  
    if (a == 0)  
        return true;  
    if (a == 1)  
        return false;  
    if (a < 0)  
        return f0(-a);  
    return f0(a - 2);  
}
```

- ▶ gibt zurück, ob a gerade ist
- ▶ Allgemeinheit: ja (alle ganzen Zahlen)
- ▶ Determiniertheit: ja (gleiche Eingabe $\rightarrow$ gleiche Ausgabe)
- ▶ Determinismus: ja (gleiche Eingabe $\rightarrow$ gleiche Schritte)
- ▶ Korrektheit: ja
- ▶ Effizienz: naja (Laufzeit wächst linear mit a)

1.1.b

```
public static boolean f1(String str1, String str2) {  
    char[] array1 = new char[Character.MAX_VALUE];  
    char[] array2 = new char[Character.MAX_VALUE];  
    for(char c : str1.toCharArray()){  
        array1[c]++;  
        // char als Index funktioniert, weil  
        // Buchstaben in Java als Ganzzahlen  
        // dargestellt werden (ASCII/Unicode)  
    }  
    for(char c : str2.toCharArray()){  
        array2[c]++;  
    }  
    for(char c = 0; c < Character.MAX_VALUE; c++){  
        if(array1[c] != array2[c])  
            return false;  
    }  
    return true;  
}
```

- ▶ gibt zurück, ob `str1` und `str2` Anagramme sind (die gleichen Zeichen enthalten)
- ▶ Allgemeinheit: ja (alle Strings)
- ▶ Determiniertheit: ja (gleiche Eingabe $\rightarrow$ gleiche Ausgabe)
- ▶ Determinismus: ja (gleiche Eingabe $\rightarrow$ gleiche Schritte)
- ▶ Korrektheit: ja (terminiert bei endlichen Strings)
- ▶ Effizienz: relativ (Laufzeit wächst mit Stringlänge und unnötig große Arrays)

1.1.c

```
public static double f2(double a, int n) {  
    double x = 1.0;  
    for (int i = 0; i < n; i++)  
        x = 0.5 * (x + a / x);  
    return x;  
}
```

- ▶ das Heron-Verfahren zur Berechnung der Quadratwurzel von a
- ▶ Allgemeinheit: ja (für positive Zahlen)
- ▶ Determiniertheit: ja (gleiche Eingabe $\rightarrow$ gleiche Ausgabe)
- ▶ Determinismus: ja (gleiche Eingabe $\rightarrow$ gleiche Schritte)
- ▶ Korrektheit: ja (terminiert bei endlichen n)
- ▶ Effizienz: naja (Laufzeit wächst linear mit n)

1.1.d

```
public static double f3(int n) {  
    double sum = 0.0;  
    for (int i = 0; i < n; i++) {  
        double x = Math.random();  
        double y = Math.random();  
        if(Math.sqrt(x*x+y*y) <= 1.0)  
            sum++;  
    }  
    return 4.0*sum/n;  
}
```

- ▶ eine Monte-Carlo-Schätzung für π
- ▶ Allgemeinheit: nein (berechnet nur eine Schätzung für π)
- ▶ Determiniertheit: ja (gleiche Eingabe $\rightarrow$ gleiche Ausgabe)
- ▶ Determinismus: nein (unterschiedliche Schritte wegen Zufallszahlen)
- ▶ Korrektheit: annähernd korrekt bei positiven n
- ▶ Effizienz: naja (Laufzeit wächst linear mit n und ungenau bei kleinen n)

1.1.e

```
public static int f4(int a, int b) {  
    if (b == 1)  
        return a;  
    return a * f4(a, b - 1);  
}
```

- ▶ berechnet a hoch b
- ▶ Allgemeinheit: ja (für alle a und positive b)
- ▶ Determiniertheit: ja (gleiche Eingabe $\rightarrow$ gleiche Ausgabe)
- ▶ Determinismus: ja (gleiche Eingabe $\rightarrow$ gleiche Schritte)
- ▶ Korrektheit: ja (terminiert bei positiven b)
- ▶ Effizienz: naja (Schleife wäre besser)

1.1.f

```
public static int f5(int a, int b) {  
    if (a == 0)  
        return b + 1;  
    if (b == 0)  
        return f5(a - 1, 1);  
    return f5(a - 1, f5(a, b - 1));  
}
```

- ▶ die Ackermann-Funktion (wird für Benchmark-Tests verwendet)
- ▶ Allgemeinheit: ja (für alle nicht-negativen a und b)
- ▶ Determiniertheit: ja (gleiche Eingabe → gleiche Ausgabe)
- ▶ Determinismus: ja (gleiche Eingabe → gleiche Schritte)
- ▶ Korrektheit: ja (terminiert bei positiven a und b)
- ▶ Effizienz: schlecht (Laufzeit wächst extrem schnell mit a und b)

1.1.g

```
public static void f6() {  
    LinkedList<Integer> list = new LinkedList<Integer>();  
    int n = 2;  
    list.add(n);  
    n = 3;  
    list.add(n);  
    while (true) {  
        n += 2;  
        boolean add = true;  
        for (Integer m : list) {  
            if (n % m == 0)  
                add = false;  
        }  
        if (add) {  
            list.add(n);  
            System.out.println(n);  
        }  
    }  
}
```

- ▶ generiert unendlich viele Primzahlen
- ▶ Allgemeinheit: schwierig zu beurteilen (der Output ist immer gleich, aber die Laufzeit ist unendlich)
- ▶ Determiniertheit: ja (gleiche Eingabe $\rightarrow$ gleiche Ausgabe)
- ▶ Determinismus: ja (gleiche Eingabe $\rightarrow$ gleiche Schritte)
- ▶ Korrektheit: partiell (generiert Primzahlen, aber terminiert nicht)
- ▶ Effizienz: nicht effizient

1.2

Warum ist die Funktion `f7()` determiniert aber nicht deterministisch?

```
public static int f7() {  
    int a = 0;  
    int b = 0;  
    if (Math.random() < 0.5) {  
        a++;  
        b++;  
    } else {  
        b++;  
        a++;  
    }  
    return a + b;  
}
```

- ▶ Ergebnis ist immer 2
 - ▶ Determiniertheit: ja (gleiche Eingabe (keine) → gleiche Ausgabe)
- ▶ durch `Math.random()` können die Schritte unterschiedlich sein
 - ▶ Determinismus: nein (die Schritte hängen vom Zufall ab)

Wiederholung Stack und Queue

Stack

- ▶ Stack $\Rightarrow$ Stapel von Elementen
 - ▶ es kann nur auf das oberste Element zugegriffen werden, das zuletzt hinzugefügt wurde
 - ▶ LIFO-Prinzip (Last In, First Out)
- ▶ Operationen:
 - ▶ `init`: erstellt einen Stack
 - ▶ `isEmpty`: prüft, ob der Stack leer ist
 - ▶ `push`: fügt Element hinzu
 - ▶ `pop`: entfernt oberstes Element (gibt es zurück)

Queue

- ▶ Queue $\Rightarrow$ Warteschlange von Elementen
 - ▶ es kann nur auf das vorderste Element zugegriffen werden, das als erstes hinzugefügt wurde
 - ▶ FIFO-Prinzip (First In, First Out)
- ▶ Operationen:
 - ▶ `init`: erstellt eine Queue
 - ▶ `isEmpty`: prüft, ob die Queue leer ist
 - ▶ `enqueue`: fügt Element am Ende hinzu
 - ▶ `dequeue`: entfernt vorderstes Element (gibt es zurück)

Aufgabe 2.1

2.1.a

Gegeben: ein Array mit ganzen Zahlen und der Länge $n \in \mathbb{N}$.

Wie lassen sich zwei Stacks aus einem Array bauen?

- ▶ ein Zeiger zeigt auf das zuletzt hinzugefügte Element im Array
 - ▶ er kann verschoben werden, wenn `push()` oder `pop()` aufgerufen wird
- ▶ da wir zwei Stacks haben, fängt der eine Stack am Anfang und der andere am Ende des Arrays an
- ▶ wird ein Element zu einem Stack hinzugefügt, wandert der jeweilige Zeiger zur Mitte des Arrays
- ▶ wenn sich beide Stacks berühren, dann kann kein Element mehr hinzugefügt werden, da das Array voll ist

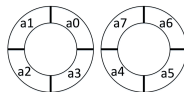
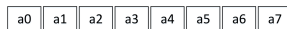


2.1.b

Gegeben: ein Array mit ganzen Zahlen und der Länge $n \in \mathbb{N}$.

Wie lassen sich zwei gleich große Queues aus einem Array bauen?

- um zwei Queues zu bauen, wird das Array in zwei Hälften geteilt



- die erste Hälfte geht von $0 \dots (n/2) - 1$ und die zweite Hälfte von $n/2 \dots n - 1$

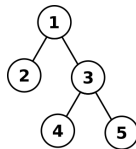
- wenn eine Queue voll ist, ersetzt das neue Element das älteste Element

- man kann sich das Array also als Ring vorstellen, in dem die Elemente nacheinander hinzugefügt werden
- um den nächsten Index zu bestimmen, wird die Formel $x \bmod n$ verwendet
 - x ist die Nummer des hinzuzufügenden Elements (beginnend bei 0)
 - n ist die Länge der Queue

Wiederholung Binärbäume

Binärbaum

- ▶ ein Baum funktioniert wie eine Liste, wobei jedes Element ein Knoten ist, der auf bis zu zwei weitere Knoten zeigt



- ▶ Baumtraversierung:

1. Tiefendurchlauf

- ▶ Preorder: Knoten $\rightarrow$ linker Unterbaum $\rightarrow$ rechter Unterbaum
(1 2 3 4 5)
- ▶ Inorder: linker Unterbaum $\rightarrow$ Knoten $\rightarrow$ rechter Unterbaum (2
1 4 3 5)
- ▶ Postorder: linker Unterbaum $\rightarrow$ rechter Unterbaum $\rightarrow$ Knoten
(2 4 5 3 1)

2. Breitendurchlauf

- ▶ Knoten werden beginnend bei der Wurzel Ebene für Ebene durchlaufen (1 2 3 4 5)

Aufgabe 2.2

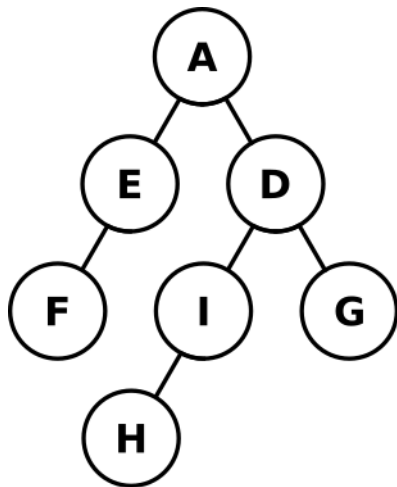
2.2.a

Gegeben:

► Preorder: A E F D I H G

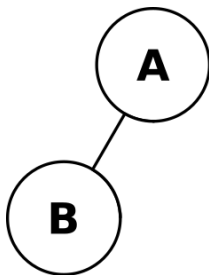
► Inorder: F E A H I D G

Geben Sie einen Baum an, zu dem die Preorder- und Inorder-Traversierung passen.

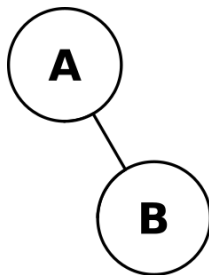


2.2.b

Gibt es zwei unterschiedliche Binärbäume, die eine gleiche Preorder- aber eine unterschiedliche Inorder-Traversierung haben?



► Preorder: A B, Inorder B A



► Preorder: A B, Inorder A B

2.2.c

Gibt es zwei unterschiedliche Binärbäume, die aber trotzdem eine gleiche Preorder- und Inorder-Traversierung haben?

- ▶ Nein, weil mit der Preorder und Inorder kann ein Binärbaum eindeutig rekonstruiert werden, wie wir es in Aufgabe 2 a gemacht haben.
- ▶ Das heißt, wenn gilt:

$$Preorder(T_1) = Preorder(T_2) \wedge Inorder(T_1) = Inorder(T_2)$$

$$\Rightarrow T_1 = T_2$$

Noch Fragen?